# ARCHITECTURE.md — Synapse Layer Complete Data Flow

## System Overview

Synapse Layer is a **Zero-Knowledge Memory Layer for AI Agents** with client-side encryption, intent validation, and trust-based conflict resolution.

### Core Principles

1. **Zero-Knowledge:** You own encryption keys; we never see plaintext
2. **Client-Side Sanitization:** PII removal before transmission
3. **Intent Validation:** Intelligent categorization with self-healing
4. **Trust Quotient™:** Adaptive conflict resolution algorithm
5. **Immutable Pipeline:** sanitize → validate → encrypt → embed → store

---

## Complete Data Flow (Mermaid Diagram)

```
graph TD
    A["Raw Text Input"] -->|user.store_memory| B["SynapseSanitizer"]

    B -->|detect & remove PII| C{PII Found?}
    C -->|Yes| D["Log Removed Items"]
    C -->|No| E["Sanitized Content"]
    D --> E

    E -->|low risk score| F["SynapseValidator"]
    E -->|high risk score| G["RISK FLAG"]
    G --> F

    F -->|classify intent| H{Intent Category}
    H -->|USER_PROFILE| I["Confidence Score"]
    H -->|MEDICAL/FINANCIAL/LEGAL| J["Auto-CRITICAL"]
    H -->|SECURITY| J
    H -->|UNKNOWN| K["Self-Healing"]

    J --> L["Validation Score"]
    I --> L
    K --> L

    L -->|score >= 0.85| M["AES-256-GCM Encryption"]
    L -->|score < 0.85| N["VALIDATION FAILED"]
    N -->|store anyway?| O{User Choice}
    O -->|proceed| M
    O -->|reject| P["Drop Memory"]

    M -->|encrypt with PBKDF2 derived key| Q["Encrypted Blob"]
    Q --> R["Generate Embeddings"]

    R -->|semantic search vector| S["pgvector Index"]
    S --> T["PostgreSQL Storage"]

    T --> U["Immutable Audit Log"]
    U --> V["Memory Successfully Stored"]

    V --> W["Return Handle & Proof"]
    W --> X["Client Application"]

    style B fill:#4f46e5,color:#fff
    style F fill:#4f46e5,color:#fff
    style M fill:#dc2626,color:#fff
    style J fill:#ea580c,color:#fff
    style T fill:#059669,color:#fff
```

---

## Component Descriptions

### 1. SynapseSanitizer

**Purpose:** Remove PII before encryption

**Patterns Detected:**
- Email: `john@example.com`
- Phone: `+55 11 99999-8888`, `(123) 456-7890`

- SSN/CPF: `123-45-6789`, `123.456.789-00`
- Credit Card: `1234-5678-9012-3456`
- API Keys: `sk_test_...`, `ghp_...`
- URLs: `https://example.com`
- IP Addresses: `192.168.1.1`

**Output:** `SanitizationResult` with:
- `sanitized_content`: Content with PII replaced with `[TYPE_REDACTED]`
- `pii_count`: Number of PII items removed
- `risk_score`: 0.0–1.0 (CRITICAL: 0.3, HIGH: 0.15, MEDIUM: 0.05 per item)
- `ner_hints`: Hints for downstream NER processing
- `is_safe`: True if risk_score < 0.05

## 2. SynapseValidator

**Purpose:** Classify intent and validate confidence

**IntentCategory Enum:**
- `USER_PROFILE` (preferences, metadata)
- `CONVERSATION` (dialogs, chats)
- `DECISION` (commitments, goals)
- `KNOWLEDGE` (facts, learning)
- `PREFERENCE` (tastes, likes/dislikes)
- `MEDICAL` (health — **auto-critical**)
- `FINANCIAL` (payments — **auto-critical**)
- `LEGAL` (contracts — **auto-critical**)
- `SECURITY` (passwords — **auto-critical**)
- `UNKNOWN` / `INVALID`

**Thresholds:**
- Confidence: 0.0–1.0
- **Valid if confidence >= 0.85** (immutable threshold)
- **Auto-critical** for MEDICAL, FINANCIAL, LEGAL, SECURITY
- **Critical keywords:** emergency, urgent, breach, attack, fraud → auto-promote

**Self-Healing:** When confidence 0.5–0.85, detect context and upgrade category

## 3. AES-256-GCM Encryption

**Key Derivation:** PBKDF2-SHA256
- Iterations: **210,000** (NIST 2023 minimum)
- Salt: 32 bytes, randomly generated
- Output: 256-bit encryption key

**Encryption Mode:**
- Algorithm: AES-256-GCM
- IV: 96 bits, randomly generated per message
- Auth Tag: 128 bits (verifies integrity & authenticity)

**Process:**

```
User Password + Random Salt → PBKDF2(210k iterations) → 256-bit Key
                                                         ↓
                                          AES-256-GCM Encryption
                                                         ↓
                                          Encrypted Blob + Auth Tag
```

## 4. Embedding Generation

**Semantic Search:**
- Convert sanitized content to vector
- Dimension: 1536 (e.g., OpenAI embeddings)
- Index: pgvector (PostgreSQL vector extension with HNSW)

## 5. PostgreSQL + pgvector

**Schema:**

```sql
CREATE TABLE memories (
    id UUID PRIMARY KEY,
    agent_id UUID NOT NULL,
    encrypted_blob BYTEA NOT NULL,        -- AES-256-GCM ciphertext
    embedding vector(1536) NOT NULL,     -- Semantic search vector
    intent_category VARCHAR(50) NOT NULL,
    confidence FLOAT NOT NULL,
    is_critical BOOLEAN NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

-- Row-Level Security (RLS)
ALTER TABLE memories ENABLE ROW LEVEL SECURITY;
CREATE POLICY agent_isolation
    ON memories
    FOR SELECT
    USING (agent_id = current_user_id());
```

**Constraints:**
- RLS (Row-Level Security) by agent_id
- Immutable audit trail
- Automatic deletion on TTL

## 6. Trust Quotient™ Calculation

```
TQ = (Recency × 0.4) + (Consistency × 0.3) + (Confidence × 0.2) + (Relevance × 0.1)

Recency: How fresh the memory (0-1)
Consistency: Agreement with other memories (0-1)
Confidence: Validation confidence (0-1)
Relevance: Semantic similarity to query (0-1)
```

**Conflict Resolution:**
- When memories conflict, highest TQ wins
- Tie-breaker: Most recent timestamp
- Audit log records all conflicts

## Pipeline Sequence (Immutable)

The following sequence is **mandatory and non-negotiable**:

1. **INPUT** → Raw text
2. **SANITIZE** → Remove PII (SynapseSanitizer)
3. **VALIDATE** → Classify intent (SynapseValidator)
4. **ENCRYPT** → AES-256-GCM with PBKDF2 key
5. **EMBED** → Generate semantic vector
6. **STORE** → Upsert to pgvector + audit log
7. **OUTPUT** → Memory handle + proof of storage

**No step can be skipped or reordered.**

---

## File Structure

```
synapse-layer/
├── sdk/
│   └── python/
│       └── synapse_memory/
│           ├── __init__.py
│           ├── sanitizer.py          ← SynapseSanitizer
│           ├── engine/
│           │   ├── __init__.py
│           │   └── validator.py      ← SynapseValidator
│           └── crypto/
│               └── __init__.py
├── docs/
│   └── ARCHITECTURE.md               ← This file
├── SECURITY.md
├── README.md
└── pyproject.toml
```

---

## Security Principles

- **Zero-Knowledge:** Client controls encryption keys; server never has plaintext
- **Client-Side Sanitization:** PII removed before leaving client
- **Immutable Audit:** Every operation logged; cannot be deleted
- **Differential Privacy:** Aggregate queries don't leak individual data
- **Handover Verification:** HMAC-SHA256 signatures on cross-model transfers

---

## Compliance & Standards

- ✅ **GDPR:** Data minimization, encryption, right to deletion
- ✅ **LGPD:** Consent, secure handling, audit trail (padrão bancário BR)
- ✅ **HIPAA:** Encryption + audit (with proper key management)

- ✅ **SOC 2:** Security controls, monitoring, incident response

---

## Performance Metrics

| Operation | Latency | Throughput |
|---|---|---|
| Sanitize | < 10ms | 1000+ items/sec |
| Validate | < 5ms | 2000+ items/sec |
| PBKDF2 (210k) | ~100ms | Single-threaded |
| AES-256-GCM | < 5ms | ~100MB/sec |
| Embedding (API) | 100-500ms | Depends on provider |
| pgvector Search | < 50ms | HNSW index |

---

End of ARCHITECTURE.md